

Objetivo

O objetivo desta atividade prática em laboratório é implementar uma **árvore binária** utilizando os conceitos de programação orientada a objetos. A estrutura deverá armazenar **strings (cadeias de caracteres)** e será reutilizada em uma aplicação prática: a representação de um **chaveamento de jogos da Copa do Mundo FIFA**.

Parte 1 – Implementação da Árvore Binária

1. Inicialmente crie os seguintes arquivos fonte:

- **NoArvoreBinaria**: classe que representa um nó da árvore;
- **ArvoreBinaria**: classe que representa a árvore binária;
- **ArvoreMain**: classe com o método principal para demonstrar o funcionamento da árvore e da aplicação prática.

2. Estrutura sugerida para a classe **NoArvoreBinaria**:

```
public class NoArvoreBinaria {  
    private String info;  
    private NoArvoreBinaria sae;  
    private NoArvoreBinaria sad;  
  
    public NoArvoreBinaria(String info,  
                            NoArvoreBinaria sae,  
                            NoArvoreBinaria sad) {  
        this.info = info;  
        this.sae = sae;  
        this.sad = sad;  
    }  
  
    // getters e setters  
}
```

3. A classe **ArvoreBinaria** deve possuir, no mínimo, um atributo para armazenar a referência à raiz da árvore:

```
private NoArvoreBinaria raiz;
```

4. Métodos a serem implementados na classe **ArvoreBinaria**:

- (a) **public ArvoreBinaria()**: construtor que instancia uma árvore vazia;

- (b) `public void defineRaiz(NoArvoreBinaria r)`: define o nó raiz principal da árvore;
 - (c) `public boolean vazia()`: retorna `true` se a árvore estiver vazia, ou `false` caso contrário;
 - (d) `public boolean pertence(String v)`: verifica se a string `v` está armazenada em algum nó da árvore;
 - (e) `public int folhas()`: retorna a quantidade de nós do tipo folha;
 - (f) `public int numNos()`: retorna a quantidade total de nós da árvore;
 - (g) `public int altura()`: retorna a altura da árvore;
 - (h) `public boolean igual(ArvoreBinaria a)`: verifica se a árvore atual e a árvore `a` têm exatamente a mesma estrutura e os mesmos valores;
 - (i) `public String imprimePre()`: retorna uma representação textual da árvore em pré-ordem;
 - (j) `public String imprimeSim()`: retorna uma representação textual da árvore em ordem simétrica;
 - (k) `public String imprimePos()`: retorna uma representação textual da árvore em pós-ordem.
5. A árvore desta atividade é uma **árvore binária geral**, e não uma árvore binária de busca. Portanto, a montagem da estrutura deve ser feita explicitamente por meio da criação dos nós e da ligação entre subárvores esquerda e direita.
6. Ao montar a árvore, lembre-se de que cada nó pode ter até duas referências:
- **sae**: subárvore da esquerda;
 - **sad**: subárvore da direita.

Um nó folha é aquele cujas referências **sae** e **sad** são `null`.

Parte 2 – Problema Aplicado: Chaveamento da Copa do Mundo FIFA

Após implementar a árvore binária, utilize-a para representar um **chaveamento eliminatório de jogos** da Copa do Mundo FIFA. Nesta modelagem, cada folha pode representar uma seleção classificada e cada nó interno pode representar uma partida do torneio.

Como a árvore desta atividade armazena **strings**, utilize diretamente nomes de seleções e identificações textuais das partidas, tornando a representação do chaveamento mais legível.

Implemente, na classe **ArvoreMain**, as seguintes funcionalidades:

1. Montar manualmente uma árvore binária que represente um pequeno chaveamento com pelo menos 4 seleções e 3 partidas;
2. Imprimir a árvore em pré-ordem, ordem simétrica e pós-ordem;
3. Informar o número total de nós do chaveamento;
4. Informar a quantidade de folhas do chaveamento;
5. Informar a altura da árvore;
6. Verificar se determinada seleção ou partida pertence à árvore;
7. Criar uma segunda árvore e testar se os dois chaveamentos são iguais.

Exemplo de contexto:

```
"Brasil"  
"Argentina"  
"Franca"  
"Alemanha"  
"Semifinal 1"  
"Semifinal 2"  
"Final"
```

Nesse exemplo, as folhas podem representar as seleções, os nós internos podem representar os jogos, e a raiz pode representar a final do torneio.

Observações

- A implementação deve representar uma árvore binária geral, sem aplicar as regras de inserção de árvore binária de busca;
- Utilize adequadamente os princípios de encapsulamento da programação orientada a objetos;
- Sempre que necessário, implemente métodos privados auxiliares recursivos para realizar as operações sobre a árvore;
- Implemente na classe `ArvoreMain` um exemplo completo demonstrando o funcionamento da árvore e da aplicação prática.

Observação Final

Após implementar todas as classes e métodos, desenvolva um programa principal que demonstre claramente o funcionamento completo da árvore binária e do chaveamento de jogos modelado sobre ela.